

Track B: Getting a Real AI Model Running Online (Backend)

Your job this week, in one sentence

Get a small AI model running, wrap it so other programs can talk to it, then put the whole thing on the internet for free — so by the end of the week, it's a real address anyone can send a message to.

Why this is the hardest part (and why that's okay)

This is genuinely the trickiest piece of the whole project. Nobody expects it to work on the first try. Budget real time for it to break — that's normal, not a sign you're doing something wrong.

A few words you'll see, explained simply

- **Ollama** — a program that lets you download and run AI models on a normal computer, and gives you a simple way to “ask” it things from code.
- **API / endpoint** — a specific web address your code can send a request to and get an answer back. Like a drive-through window for your program.
- **CORS** — a browser security rule that blocks a webpage from talking to a different address unless that address explicitly allows it. You'll hit this error at least once; it has a one-line fix.
- **Docker / container** — think of it as a shipping container for your code. It packs your app and everything it needs into one box, so it runs the same way on any computer — including Hugging Face's servers.
- **Hugging Face Space** — a free website that can run your Docker container for you, for free, and gives it a public web address.

Two parts to this track

Part 1 — Get the model talking (through Flask)

- 1. Install Ollama and download a model.** Run `ollama pull qwen3:4b`, then test it with `ollama run qwen3:4b` — type something and see if it answers. If that works, the hardest local step is already done.
- 2. Set up a small Python web server (Flask).** Create a virtual environment, then install `flask`, `flask-cors`, and `requests`.
- 3. Build one route: `/chat`.** It should take a message, pass it to Ollama, and send back Ollama's answer piece by piece as it's generated — not all in one go. This matches the “streaming” the frontend is already built to expect.
- 4. Test it without a browser first.** Use a terminal command (`curl`) to send a fake message and confirm the reply arrives gradually, not as one big block of text.

5. Turn on CORS. Add flask-cors with one line so a webpage will actually be allowed to talk to your server.

6. Write down how to run it. A few lines is enough: “run Ollama, then run this file, then it’s live on this address.” You’ll need this yourselves once Part 2 starts.

Part 2 — Put it on the internet, for free

1. Look at a working example before starting from scratch.

Search GitHub for “ollama huggingface space docker” — real projects already do exactly this. Copy the structure, don’t reinvent it.

2. Write a Dockerfile. A short text file that says: “start with Python, install Ollama, copy in the app, and run it on port 7860” (that exact port is what Hugging Face expects).

3. Write a startup script. Something that starts Ollama, waits for it to be ready, downloads the model, and only then starts the Flask app — in that order.

4. Create a Hugging Face Space. Go to huggingface.co → New Space → choose “Docker.” Push your code to it, the same way you’d push to GitHub.

5. Expect it to fail the first couple of times — and fix it. Common issues: wrong port number, the model taking too long to download during setup, or a permissions error. This step alone can eat half your week’s hours. That’s expected, not a failure.

6. Confirm it actually works from outside Hugging Face’s own page. Send it a real request from your own laptop, not just their built-in test box.

What “done” looks like by the end of the week

- ☐ Ollama answers a question in your terminal.
- ☐ Flask’s /chat route sends back a real answer, in pieces, when tested with curl.
- ☐ The whole thing is deployed on a Hugging Face Space with a public web address.
- ☐ That public address answers a message sent from your own laptop.

A heads-up on speed

The free version of this only uses a normal processor, not a graphics card, so answers will feel slow — maybe 10 to 20 seconds to fully arrive. That’s expected and completely fine for a class project. It’s also exactly why “streaming” (showing the answer as it’s typed, rather than waiting for the whole thing) matters — it makes the wait feel much shorter than it actually is.